

# MASTER TECHNICAL KNOWLEDGE BASE

Unified Engineering Manual: Security, AI Agents, Full-Stack & Systems

Published: September 30, 2026 | Version 1.0 | Source-Grounded Architecture Manual

**Executive Summary & Knowledge Base Architecture:**

This master document consolidates 56 primary sources across four critical engineering domains: **Zero-Trust IAM & Security Architecture**, **AI Engineering & Model Context Protocol (MCP)**, **Full-Stack TypeScript & Async Python Reference**, and **Technical Monetization Playbooks**. All specifications, cryptographic parameters, protocol flows, and code patterns are grounded directly in enterprise documentation.

| Module   | Domain & Core Specifications                                                                                                                                                       |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Module 1 | <b>Zero-Trust Security, Identity &amp; Advanced IAM Architecture</b><br>RFC 7636 PKCE, RFC 9700 OAuth 2.1, M2M Private Key JWT, DPoP, Refresh Token Rotation, In-Database RBAC     |
| Module 2 | <b>AI Engineering, Agentic Architecture &amp; Model Context Protocol</b><br>MCP Client/Server JSON-RPC, Skills vs MCP, LangGraph Stateful SDLC, Vector Search & Docker pgvector    |
| Module 3 | <b>Full-Stack Engineering, TypeScript &amp; Python Reference</b><br>4-Pillar TypeScript Spiral, Utility Types & Standalone IAM Helpers, AsyncIO/asyncpg, Pydantic & Ollama         |
| Module 4 | <b>Business, Side Hustles, AI Productivity &amp; Marketing Playbooks</b><br>\$40-\$80+/hr Code Evaluation Tracks, Vibe-to-Production Remediation, Daily Schedules, Mindstream SOPs |

A consolidated, production-grade reference manual synthesizing technical architecture, identity & security specifications, AI engineering practices, full-stack development, and monetization playbooks grounded in the notebook sources.

## MODULE 1: ZERO-TRUST SECURITY, IDENTITY & ADVANCED IAM ARCHITECTURE

### 1.1 OAuth 2.0 PKCE (RFC 7636) & OAuth 2.1 / RFC 9700 Security Mandates

- **The Modern Security Mandate:** Modern Identity and Access Management (IAM) requires transitioning away from static shared secrets toward dynamic, cryptographic proof-of-possession. Legacy patterns like the Implicit Flow and Resource Owner Password Credentials (ROPC) are deprecated under OAuth 2.1 and RFC 9700 (OAuth 2.0 Security Best Current Practice) because front-channel token delivery exposes sensitive artifacts to browser history leaks, Referer header leakage, and unauthorized interception.

- **Authorization Code Interception Attack:** Public clients (Single-Page Applications, mobile apps) cannot securely store static secrets. Front-channel codes are vulnerable to custom URI scheme collisions, open redirectors, compromised browser extensions, and network interception.

- **Cryptographic Mechanics:**

**code\_verifier:** A high-entropy, cryptographically random string between 43 and 128 characters using URL-safe characters (**A-Z**, **a-z**, **0-9**, **-**, **.**, **\_**, **~**).

**code\_challenge:** Derived value sent in the front-channel request: **BASE64URL(SHA256(ASCII(code\_verifier)))** without Base64 padding.

**code\_challenge\_method:** Must strictly enforce **S256**. The **plain** method is insecure and must be explicitly rejected by modern Authorization Servers.

*Canonical Test Vector (RFC 7636):*

**code\_verifier:** **dBjftJeZ4CVP-mB92K27uhbUJU1p1r\_wW1gFWFOEjXk**

**code\_challenge\_method:** **S256**

**code\_challenge:** **E9MeIhoa20wvFrEMTJguCHaoeK1t8URWbuGJSstw-cM**

- **Protocol Flow:**

1. *Client Generation:* Client generates **code\_verifier** and computes **code\_challenge** via S256.

2. *Front-Channel Request:* Client redirects user to **/authorize** with **code\_challenge** and **code\_challenge\_method=S256**.

3. *Authorization Code Issuance:* Authorization Server (AS) authenticates user and issues single-use code.

4. *Back-Channel Exchange:* Client presents code and unhashed **code\_verifier** to **/token**. AS recomputes SHA-256 hash and verifies match using constant-time comparison (**timingSafeEqual**).

- **Defense-in-Depth for Confidential Clients:** OAuth 2.1 mandates PKCE for both public and confidential clients. This ensures that even if a long-lived **client\_secret** leaks via source control or logs, an intercepted authorization code remains cryptographically unusable.

### 1.2 Machine-to-Machine (M2M) Authentication & Sender-Constraining

- **Client Credentials Mandate (RFC 6749 Section 4.4.3):** Authorization Servers **MUST NOT** issue refresh tokens for Client Credentials grants. Since clients possess asymmetric or long-lived credentials, issuing refresh tokens adds unnecessary

state and attack surface.

- **Transitioning from `client_secret`:** Static shared secrets leak in git histories, CI/CD logs, and APM tools. They lack proof-of-possession and are difficult to rotate.

- **Private Key JWT (`private_key_jwt` - RFC 7523):**

Client generates an RSA 2048+ or EC P-256 keypair.

Registers public key with IdP (via static JWKS or `jwks_uri`).

Signs short-lived client assertion JWT (1-2 min expiration) containing `iss`, `sub` (= `client_id`), `aud` (token endpoint URL), and unique `jti` for replay protection.

- **mTLS (`tls_client_auth` - RFC 8705):** Identity is proven during the TLS handshake via X.509 client certificate. The access token is bound to the certificate hash via the `cnf.x5t#S256` claim.

- **Demonstrating Proof-of-Possession (DPoP - RFC 9449):**

Cryptographically binds access tokens and refresh tokens to an ephemeral client keypair at the application layer.

DPoP proof JWT contains `htm` (HTTP method), `htu` (HTTP URI), `iat`, `jti` (replay cache), and optional server `nonce`. Access token contains `cnf.jkt` thumbprint claim.

### 1.3 Refresh Token Rotation (RTR) & Session Hardening

- **Refresh Token Rotation:** Public clients receive a single-use refresh token alongside a new access token on every refresh. The prior refresh token is invalidated immediately.

- **Breach & Reuse Detection:** If a previously consumed refresh token is presented, the Authorization Server revokes the entire token family (`family_id`), invalidates active sessions, and emits security alerts.

- **SPA Security Architecture:** SPAs should avoid storing refresh tokens in `localStorage`, `sessionStorage`, or `indexedDB` due to XSS risks. Recommended pattern is Backend-for-Frontend (BFF) holding `HttpOnly`, `Secure`, `SameSite` cookies, or keeping tokens strictly in memory.

- **Pushed Authorization Requests (PAR - RFC 9126):** Moves front-channel authorization parameters to a direct back-channel `/par` POST request before redirection, preventing parameter tampering and Referer header leaks.

- **Resource Indicators (RFC 8707):** Explicitly binds token issuance to target Resource Servers via `resource` parameter, setting explicit `aud` claims in access tokens.

## MODULE 2: AI ENGINEERING, AGENTIC ARCHITECTURE & MODEL CONTEXT PROTOCOL (MCP)

### 2.1 Model Context Protocol (MCP) Standards & Architecture

- **Protocol Core:** Open client-server standard introduced by Anthropic (using JSON-RPC 2.0 over STDIO or HTTP/SSE) to connect AI models safely to enterprise tools and data sources.

- **Three Core Abstractions:**

1. **Resources:** Passive read-only data endpoints (files, database records, API outputs, ontologies).

2. **Tools:** Executable functions/actions called by agents (API invocations, code runners, database operations) with explicit input schemas.

3. **Prompts:** Pre-defined prompt templates and agent role definitions.

- **FastMCP & Identity-Aware Tool Calling:** FastMCP provides high-level Python bindings. Gateway endpoints accept Google/Auth0 ID tokens (`auth_token`), verifying JWT claims and enforcing role-based tool execution at runtime.

## 2.2 Skills vs. MCP Servers

- **Metaphor & Layering:**

**Skill ("Sheet Music / Playbook"):** Procedural knowledge layer containing step-by-step Standard Operating Procedures (SOPs), domain judgment, and workflow logic in a **SKILL.md** file. Runs inside the agent's LLM context window.

**MCP Server ("Instrument / Connectivity"):** Wiring/connectivity layer providing typed tool calls, database connections, and external API integrations over JSON-RPC. Runs as a separate process outside the agent's LLM.

- **Progressive Disclosure:**

Flat MCP tool definitions can consume up to 72% of context windows upfront.

Skills use progressive disclosure: agents load metadata (~80 tokens) initially, fetching the full **SKILL.md** procedural text only when task triggers match.

- **Hard Enforcement Rule:** Instructions in a **SKILL.md** guide behavior, but cannot enforce hard security boundaries. Hard security, authorization, rate limits, and audit logs must be enforced deterministically in code (MCP server, API gateway, or PolicyEngine).

## 2.3 Autonomous SDLC Harnesses & Stateful Agent Loops

- **LangGraph State Machine Lifecycle:**

1. **initialize:** Parses GitHub webhook (**SDLCEvent**), creates isolated feature branch **feature/AGENT-{id}**.

2. **propose\_patches:** Agent generates structured **PatchProposal** objects.

3. **apply\_patches:** Writes files in sandbox with path-traversal guards (rejects `..`, absolute paths, `.git`).

4. **audit\_patches:** Non-bypassable **PolicyEngine** static scan for secret leakage (AKIA, ghp\_) and forbidden AST nodes (`eval`, `exec`, `__import__`). Violations abort workflow to **notify\_security**.

5. **run\_tests:** Non-blocking test execution (`pytest -q`).

6. **check\_test\_results** (Retry Loop):

*Pass \$*

igtarrow\$ **finalize** (Git commit, push, GitHub PR creation).

*Fail & attempts < max \$*

igtarrow\$ **repair\_patches** (feeds truncated 6,000-char traceback back to agent, loops to **apply\_patches**).

*Fail & max attempts reached \$*

igtarrow\$ **END**.

- **Sandboxed Execution:** Ephemeral Docker/gVisor containers with `--network none`, `--memory 2g`, `--cpus 2.0`, `--cap-drop=ALL`, and read-only root mounts prevent process escapes, fork bombs, and network leaks during test runs.

## 2.4 Production RAG & Vector Search Optimization

- **In-Database Zero-Trust RBAC:**

*Anti-Pattern:* Fetching top-\$k\$ vectors first, then filtering unauthorized docs in Python. Causes context starvation, timing side-channels, and pagination corruption.

*Shift-Left Solution:* Authority evaluated directly inside PostgreSQL WHERE clause before data leaves disk or enters database buffers.

SQL Query:

```
SELECT id, title, content, 1 - (embedding <=> $1::vector) AS similarity
FROM enterprise_documents
WHERE allowed_roles ?| $2::text[]
ORDER BY embedding <=> $1::vector
LIMIT $3;
```

Uses `?|` (JSONB overlap operator) backed by a GIN index on `allowed_roles`, combined with an HNSW index (`vector_cosine_ops`) for `$O(\log N)$` ANN search.

- **Context Starvation & Fragment-Scored Allocator:** Query decomposition splits multi-intent queries into sub-queries, but relevance-greedy packers starve mid-scoring sub-intents. Reserving a 1-passage floor per sub-intent scored against its own fragment query increases sub-intent coverage to 89.4%.

## 2.5 PostgreSQL + pgvector Docker Deployment & Infrastructure Commands

### • Single-Line Container Deployment:

```
docker run -d --name pgvector -e POSTGRES_USER=rag_user -e POSTGRES_PASSWORD=rag_password -e POSTGRES_DB=enterprise_rag
```

(Matches connection string: `postgresql://rag_user:rag_password@localhost:5432/enterprise_rag`).

### • Enable Vector Extension:

```
docker exec -it pgvector psql -U rag_user -d enterprise_rag -c "CREATE EXTENSION IF NOT EXISTS vector;"
```

### • Container Management Commands:

```
docker ps                # Check container status
docker stop pgvector     # Stop container (keeps volume data intact)
docker start pgvector     # Restart container
docker rm -f pgvector    # Remove container (data volume persists)
```

# MODULE 3: FULL-STACK ENGINEERING, TYPESCRIPT & PYTHON REFERENCE

## 3.1 TypeScript Mastery & Type System Mechanics

### • Spiral Learning Model (4 Pillars):

1. *Learn*: Direct instruction (Total TypeScript).
2. *Reference*: Declarative handbook knowledge (TS Handbook).
3. *Practice*: Deliberate retrieval practice (type-challenges).
4. *Retain*: Spaced repetition (Execute Program).

### • Five Core Mental Models:

1. *Types are erased*: Static type annotations do not exist at runtime.
2. *Structural typing*: Compatibility based on shape, not nominal declaration.
3. *Inference first, annotation second*: Annotate function parameters and public API boundaries; let TypeScript infer local variables.
4. *Narrowing as control flow*: Types narrow based on conditional branches (`typeof`, `instanceof`, `in`, equality).
5. *any as an escape hatch*: `any` disables type checking and propagates unsafety.

### • Key Patterns:

**as const + Union Pattern** (replaces enums):

```
const ROLES = ["admin", "user", "guest"] as const;
type Role = typeof ROLES[number]; // "admin" | "user" | "guest"
```

**Discriminated Unions for State:**

```
type AsyncState<T> =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "success"; data: T }
  | { status: "error"; error: Error };
```

## 3.2 TypeScript Utility Types & Standalone IAM Practice Implementation

```
import { randomBytes, createHash, timingSafeEqual } from 'node:crypto';

// 1. Cryptographic PKCE Helper (RFC 7636)
export class PKCEHelper {
  public static generatePKCEPair(length: number = 64) {
    if (length < 43 || length > 128) throw new RangeError('Verifier length 43-128');
    const unreserved = 'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789-._~';
    const bytes = randomBytes(length);
    let codeVerifier = '';
    for (let i = 0; i < length; i++) codeVerifier += unreserved[bytes[i] % unreserved.length];
    const codeChallenge = createHash('sha256').update(codeVerifier, 'utf8').digest('base64url');
    return { codeVerifier, codeChallenge, method: 'S256' as const };
  }

  public static verifyCodeChallenge(verifier: string, storedChallenge: string): boolean {
    const computed = createHash('sha256').update(verifier, 'utf8').digest('base64url');
    const bufA = Buffer.from(computed, 'utf8');
    const bufB = Buffer.from(storedChallenge, 'utf8');
    return bufA.length === bufB.length && timingSafeEqual(bufA, bufB);
  }
}

// 2. JWT Claim Verification & Tenant Guard
export interface EnterpriseJWTPayload {
  iss: string; sub: string; aud: string | string[]; exp: number; tenant_id: string; roles: string[];
}

export class JWTClaimValidator {
  public static validateClaims(rawToken: string, expectedIss: string, expectedAud: string): EnterpriseJWTPayload {
    const parts = rawToken.split('.');
    if (parts.length !== 3) throw new Error('Invalid JWT format');
    const payload: EnterpriseJWTPayload = JSON.parse(Buffer.from(parts[1], 'base64url').toString('utf8'));
    const now = Math.floor(Date.now() / 1000);
    if (now >= payload.exp) throw new Error('Token expired');
    if (payload.iss !== expectedIss) throw new Error('Invalid issuer');
    const audMatch = Array.isArray(payload.aud) ? payload.aud.includes(expectedAud) : payload.aud === expectedAud;
    if (!audMatch) throw new Error('Invalid audience');
    if (!payload.tenant_id) throw new Error('Missing tenant_id claim');
    return payload;
  }
}
```

## 3.3 Python AI Development & Async Architecture

- **AsyncIO & Connection Pooling:** Non-blocking asynchronous I/O via **asyncio** and **asyncpg** prevents thread starvation under high RAG concurrency. Connection pools must be loop-aware to handle process restarts cleanly.

- **Pydantic DTO Contracts:** Strict schema validation enforces deterministic inputs/outputs across LLMs, vector search, and API boundaries.
- **Ollama Local GPU Execution:** Docker Compose setups require `deploy.resources.reservations.devices` with `driver: nvidia` for GPU acceleration. Models default to `/root/.ollama/models` inside containers.

## MODULE 4: BUSINESS, SIDE HUSTLES, AI PRODUCTIVITY & MARKETING PLAYBOOKS

### 4.1 High-Yield Technical Income & AI Code Evaluation

- **Hourly AI Code Review & RLHF Tracks (\$40–\$80+/hr):**
  - DataAnnotation.tech (Coding Track):** \$40–\$60/hr. Asynchronous auditing of Python/C#/SQL LLM completions for edge cases, race conditions, and vulnerabilities.
  - Turing (LLM Code Reviewer):** \$40–\$80+/hr. Debugging and scoring enterprise model code; fast-tracked via DSA/architecture diagnostics.
  - Outlier.ai / Invisible:** \$40–\$65/hr. Multi-turn reasoning evaluation, adversarial prompt construction, and RLHF scoring.
- **"Vibe-to-Production" Codebase Remediation (\$65–\$110/hr):**

Solo builders using Cursor, Bolt.new, v0, or Replit hit hard walls with unhandled async loops, connection pool exhaustion, schema drift, and auth bugs.

Positioning: Fixed-rate or hourly "Codebase Doctor" offering async loop fixes, Pydantic schema hardening, and zero-trust Auth0/PostgreSQL security.

### 4.2 Daily Execution Schedule & Revenue Strategy

| Time Block    | Operational Focus & Primary Objective                   |
|---------------|---------------------------------------------------------|
| 07:30 - 11:00 | Primary Job Search & Target Sniper Outreach (Protected) |
| 11:00 - 12:30 | Systems Architecture & Open-Source Portfolio Build      |
| 13:30 - 16:30 | Hourly AI Code Review & Client Code Remediation (\$/hr) |
| 16:30 - 17:00 | Daily Pipeline, Billable Logging & Target Staging       |

### 4.3 Time Reclamation & Operations Playbooks (Mindstream & HubSpot)

- **Mindstream 285 Prompts Playbooks:**
  - Email Domination System:** Tame inbox, batch responses, and cut email processing time in half.
  - SOP Generator Toolkit:** Document processes so no operational knowledge depends on a single person.
  - Meeting Minimizer Protocol:** Eliminate unnecessary meetings and condense essential syncs to 15 minutes.
  - Hiring & Delegation Accelerator:** Standardize onboarding and delegate routine tasks safely.
- **Marketing & Sales Productization:**

Productizing consulting into fixed-deliverable tiers.

Lead magnet launch funnels and 1000+ HubSpot x Mindstream prompt assets for SEO, email, and social campaigns.